CS 412 / 512
Introduction to Computer Graphics

Zhu Wang
Assistant Professor of Computer Science



COMPUTER SCIENCE

WebGL

Rendering

```
gl.drawElements(mode, count, type, offset);
```

```
Vertices: [
    -1, -1, -1,   // 0
     1, -1, -1,   // 1
     1,  1, -1,   // 2
    -1,  1, -1,   // 3
    -1, -1,  1,   // 4
     1, -1,  1,   // 5
     1,  1,  1,   // 6
    -1,  1,  1    // 7
  ]
```

```
indices :[
  // Front
  4, 5, 6,    4, 6, 7,
  // Back
  1, 0, 3,    1, 3, 2,
  // Top
  3, 7, 6,    3, 6, 2,
  // Bottom
  0, 1, 5,    0, 5, 4,
  // Right
  1, 2, 6,    1, 6, 5,
  // Left
  0, 4, 7,    0, 7, 3,
]
```

Draw vertices in the order defined in the **indices** array

Mode: gl.TRIANGLES

Count: indices.length

Type: gl.UNSIGNED_SHORT

Offset: 0

Interactive Controls

Controlling camera/object movements

Absolute inputs

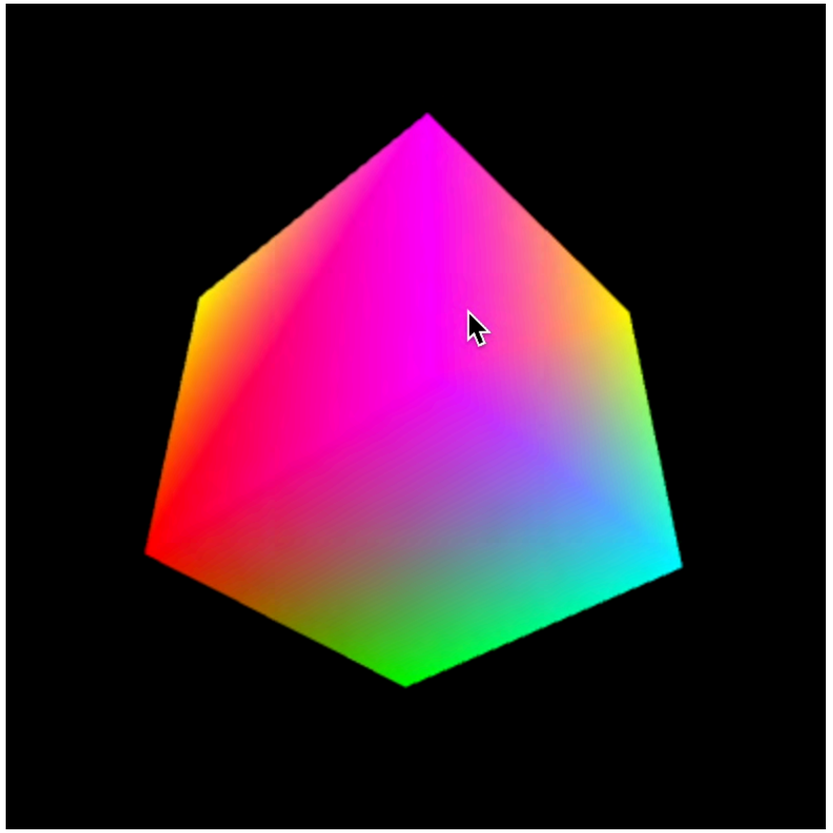
Direct positions

e.g. tablet, touch screen

Relative/incremental inputs

Delta positions (changes since last frame)

e.g. mouse, trackball, joystick



```
canvas.addEventListener('mousemove', e=>{
  if(!mouseDown) return;
  let dx = e.clientX - lastX;
  let dy = e.clientY - lastY;
  cubeRotY += dx*0.01;
  cubeRotX += dy*0.01;
  lastX = e.clientX; lastY = e.clientY;
});
```

Object-to-clip Transformation

```
gl_Position = uProjectionMatrix * uModelViewMatrix * vec4(aPosition,1.0);
```

```
uModelViewMatrix
```

```
document.addEventListener('keydown', e=>{
  const step=0.2;
  switch(e.key){
    case 'ArrowUp': camY -= step; break;
    case 'ArrowDown': camY += step; break;
    case 'ArrowLeft': camX += step; break;
    case 'ArrowRight': camX -= step; break;
    case 'w': camZ += step; break;
    case 's': camZ -= step; break;
  }
});
```

```
  // init model-view matrix as identity matrix
let modelViewMatrix = mat4Identity();
modelViewMatrix = mat4Translate(modelViewMatrix, [camX, camY, camZ]);
modelViewMatrix = multiply4(rotation, modelViewMatrix);
```

Object-to-clip Transformation

`uProjectionMatrix`

Canonical perspective

$$M_p = \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & f+n & -fn \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Standard perspective for WebGL

$$\longrightarrow \quad M_p{}' = \begin{bmatrix} \frac{f_y}{\alpha} & 0 & 0 & 0 \\ 0 & f_y & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

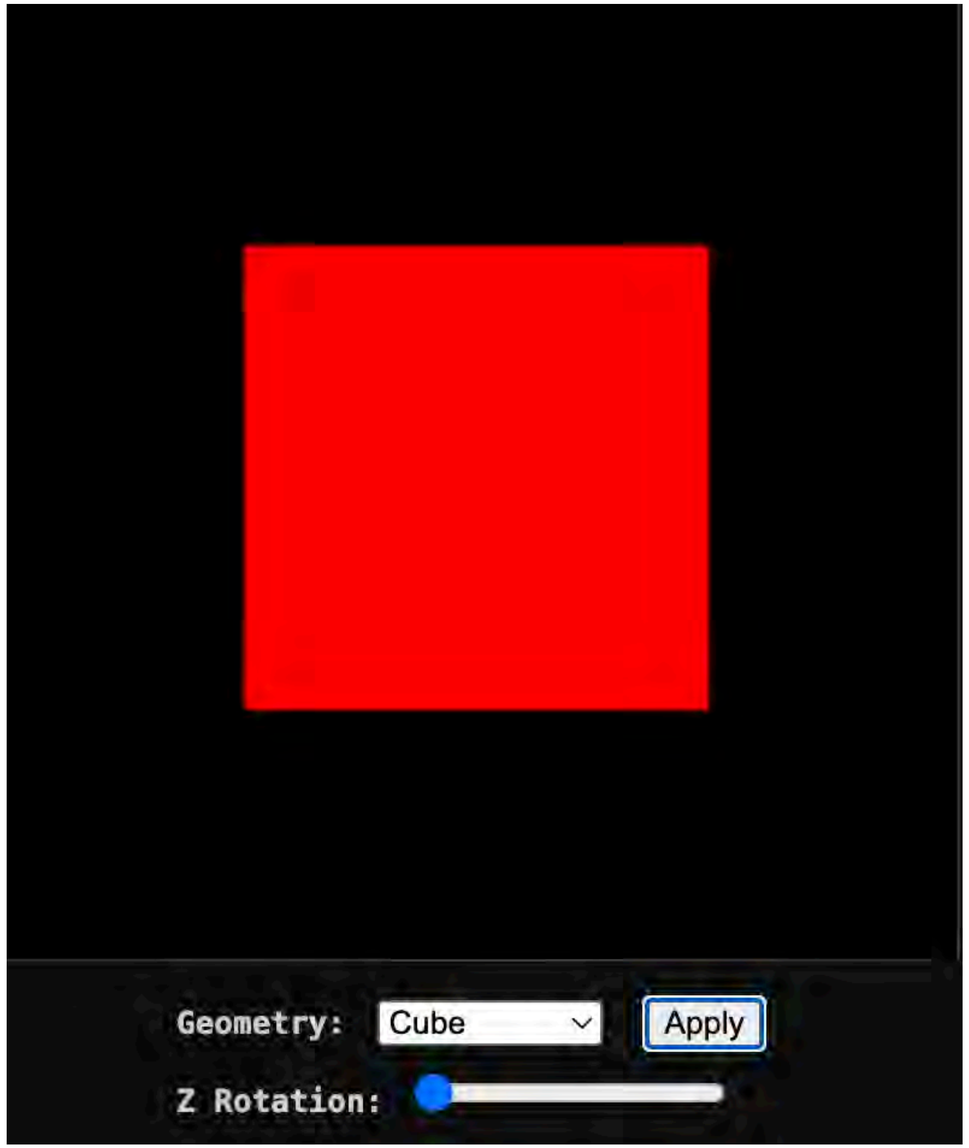
Vertical field-of-view θ

$$f_y = \frac{1}{tan(\frac{\theta}{2})}$$

$$\alpha = \frac{width}{height}$$

UI Elements

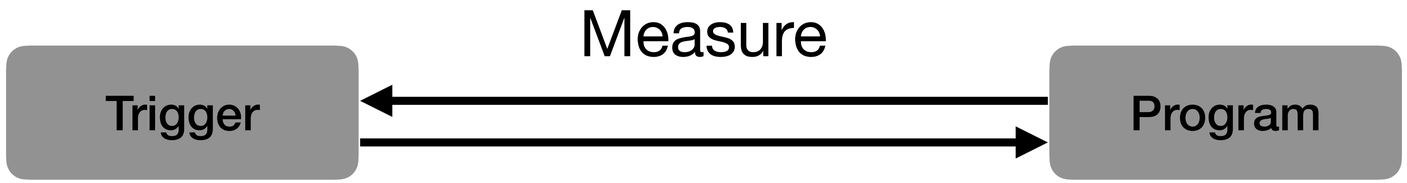
Button, slider, menu



Events

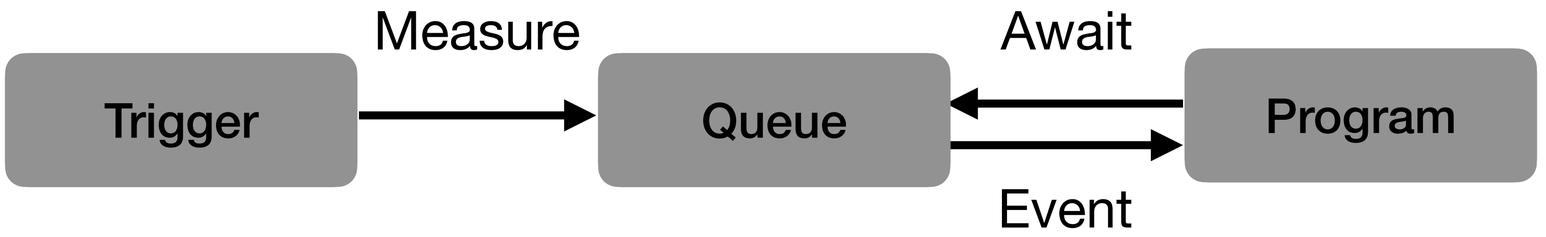
Immediate Mode

e.g. Program asks the key state each frame



Event Queue Mode

e.g. Interacting with GUI buttons, sliders



Callbacks

Functions that response to the events

```
canvas.addEventListener(
  'mousedown', e=>{mouseDown=true;
  lastX=e.clientX;
  lastY=e.clientY;});

canvas.addEventListener(
'mouseup', ()=>mouseDown=false);

canvas.addEventListener(
 ‘mousemove', e=>{
  if(!mouseDown) return;
  let dx = e.clientX - lastX;
  let dy = e.clientY - lastY;
  cubeRotY += dx*0.01;
  cubeRotX += dy*0.01;
  lastX = e.clientX;
  lastY = e.clientY;
});
```

```
document.addEventListener(
  'keydown', e=>{const step=0.2;
  switch(e.key){
    case 'ArrowUp': camY -= step; break;
    case 'ArrowDown': camY += step; break;
    case 'ArrowLeft': camX += step; break;
    case 'ArrowRight': camX -= step; break;
    case 'w': camZ += step; break;
    case 's': camZ -= step; break;
  }});
    document.getElementById(“slider")
                .addEventListener("input", e => {
      sliderValue = e.target.value;
    });
    document.getElementById(“button1")
              .onclick = function () {
      isRotate = !isRotate;
    }
window.onload = function() {
  setInterval(render, 30);
}

requestAnimationFrame(draw)
```

Scheduling Updates

SetInterval() vs requestAnimationFrame()

```
window.onload = function() {
  setInterval(render, 30);}
```

- Fixed timer; independent to actual frame rate
- Need to manually stop it when no more animation or browser if not active
- Interval 0 means as fast as possible

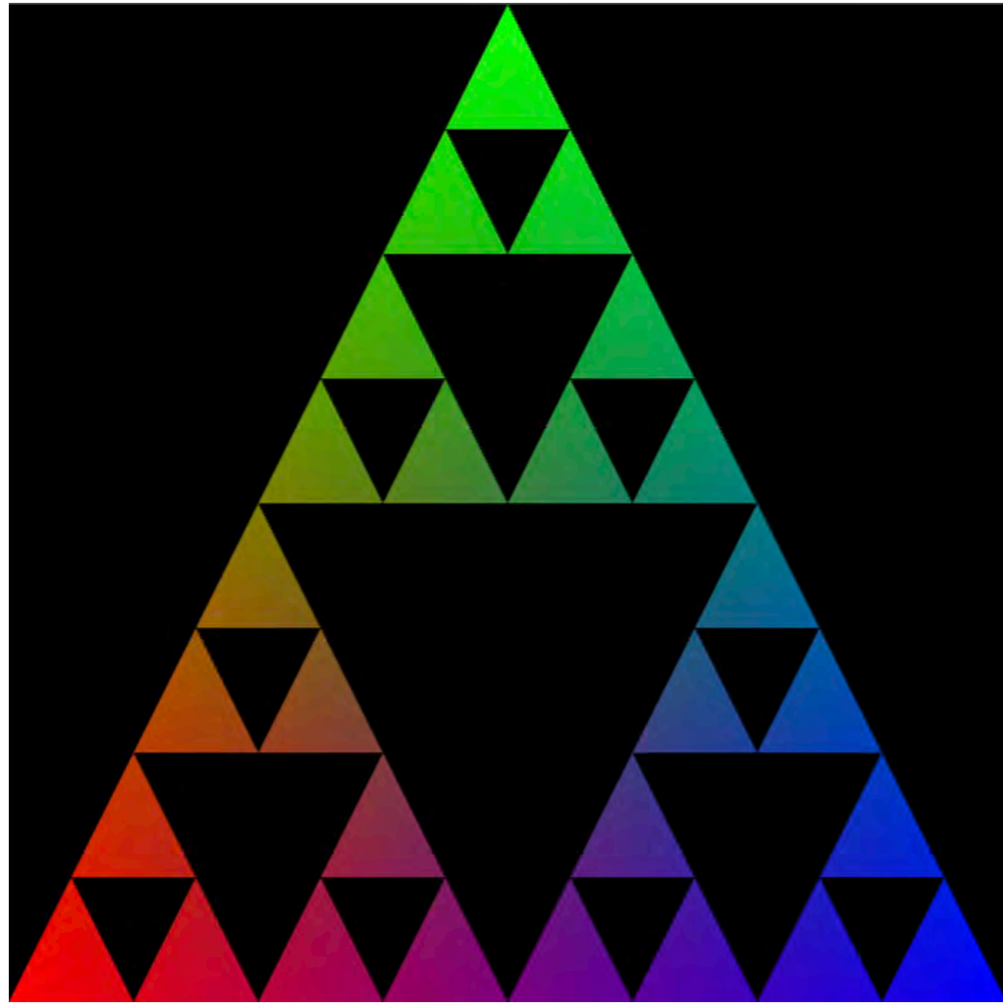
```
requestAnimationFrame(render)
```

- Schedules animation callback to run right before the browser's next repaint
- Automatically paused when inactive
- Smooth; synced with display refresh rate

Fractals

The Sierpiński Gasket

- Start with one big triangle;
- At each recursion, find midpoints on all edges using the midpoints;
- Use the midpoints to subdivide the triangle into four smaller triangles;
- Repeat the process until the recursion reaches the target depth;
- At the final level of recursion, record the vertices and colors for rendering.



Hierarchical Modeling

Hierarchical Modeling

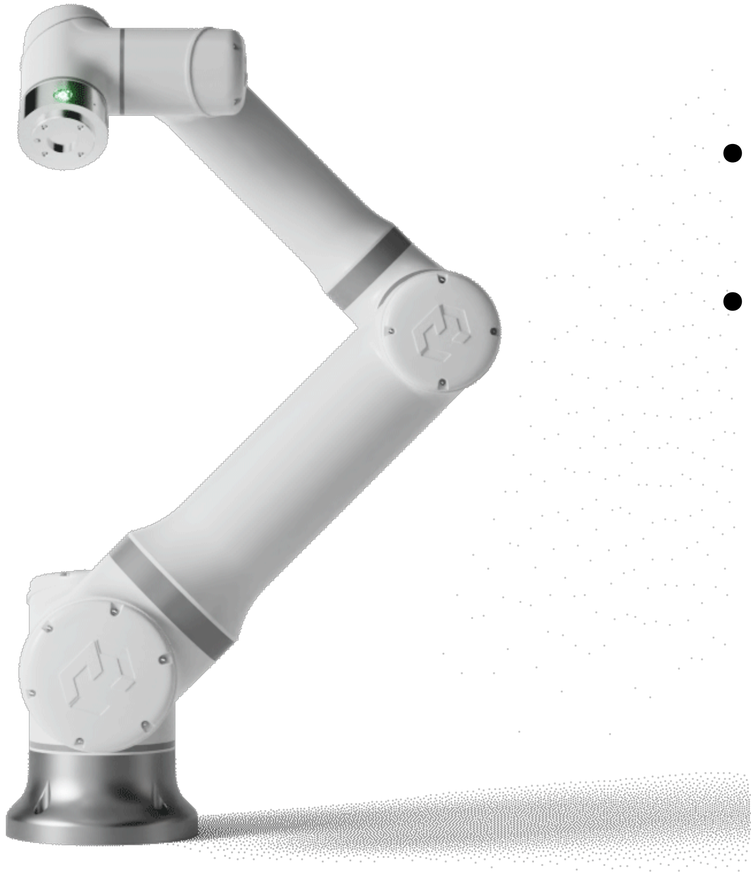
Articulated model

- Build complex models by combining simple geometric primitives

- Drive the motion of each part using chained transformation matrices in a hierarchy (tree structure)



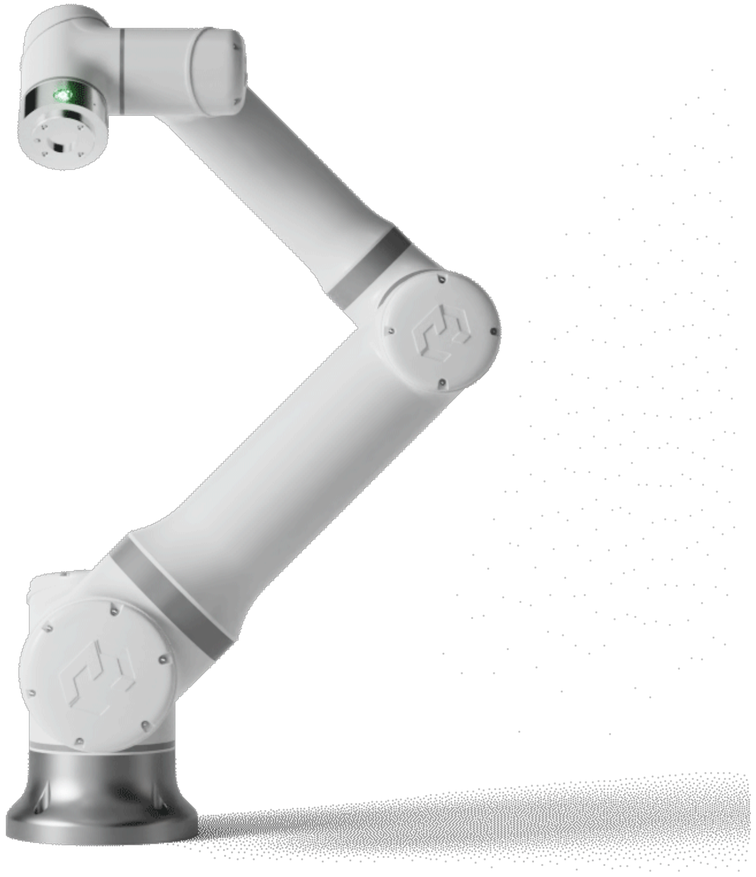
Hierarchical Modeling

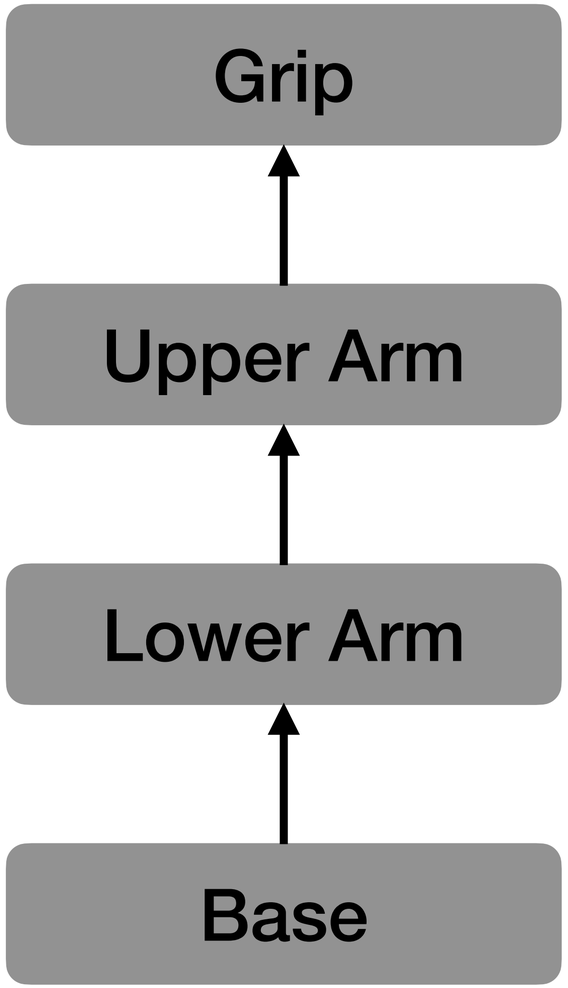


- Rigid parts linked by joints
- Each part can move or rotate relative to the others using joints or hinges

Hierarchical Modeling

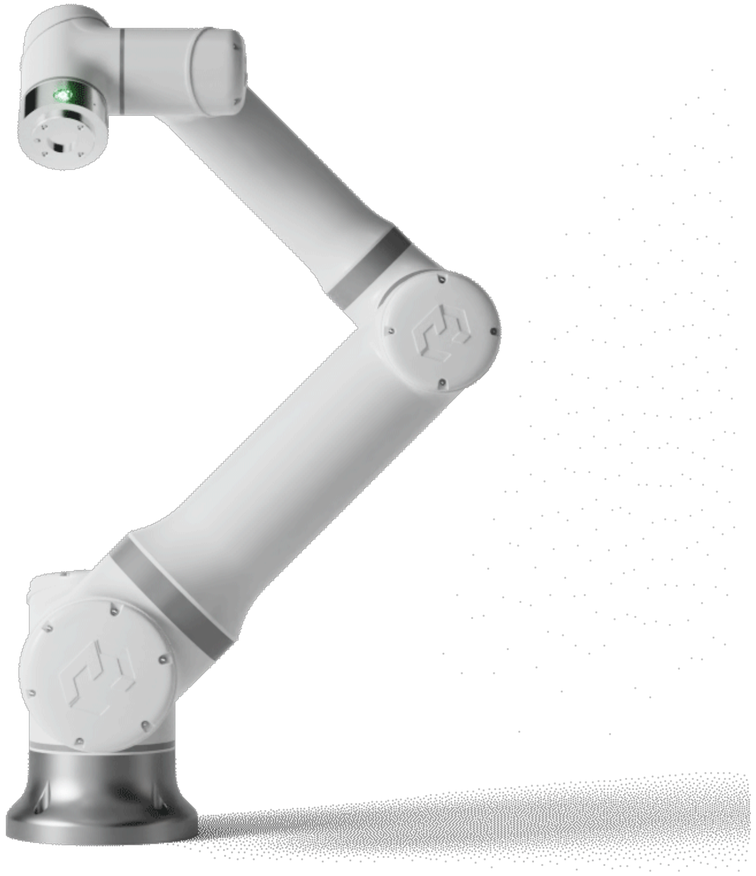
Local & World coordinates





$$M_{world}(child) = M_{world}(parent) \cdot M_{local}(child)$$

Hierarchical Modeling



$$M_{base} = T_{base} \cdot R_{base}$$

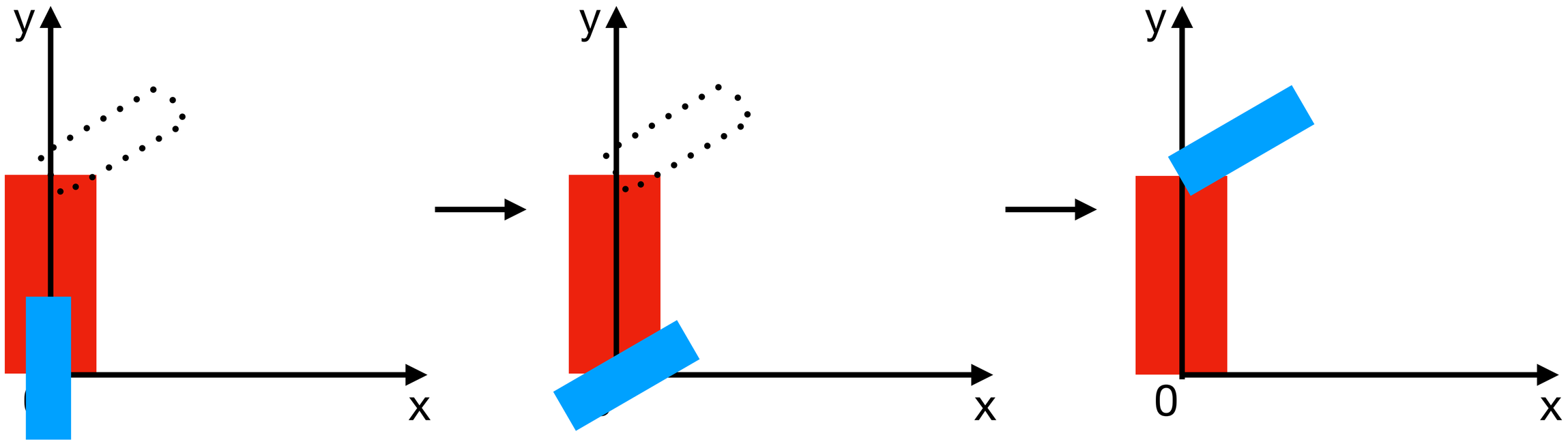
$$M_{lower} = M_{base} \cdot T_{lower} \cdot R_{lower}$$

$$M_{upper} = M_{lower} \cdot T_{upper} \cdot R_{upper}$$

$$M_{grip} = M_{upper} \cdot T_{grip} \cdot R_{grip}$$

Independent Transformations

World coordinate frame



$$p' = T \cdot R \cdot S \cdot p$$

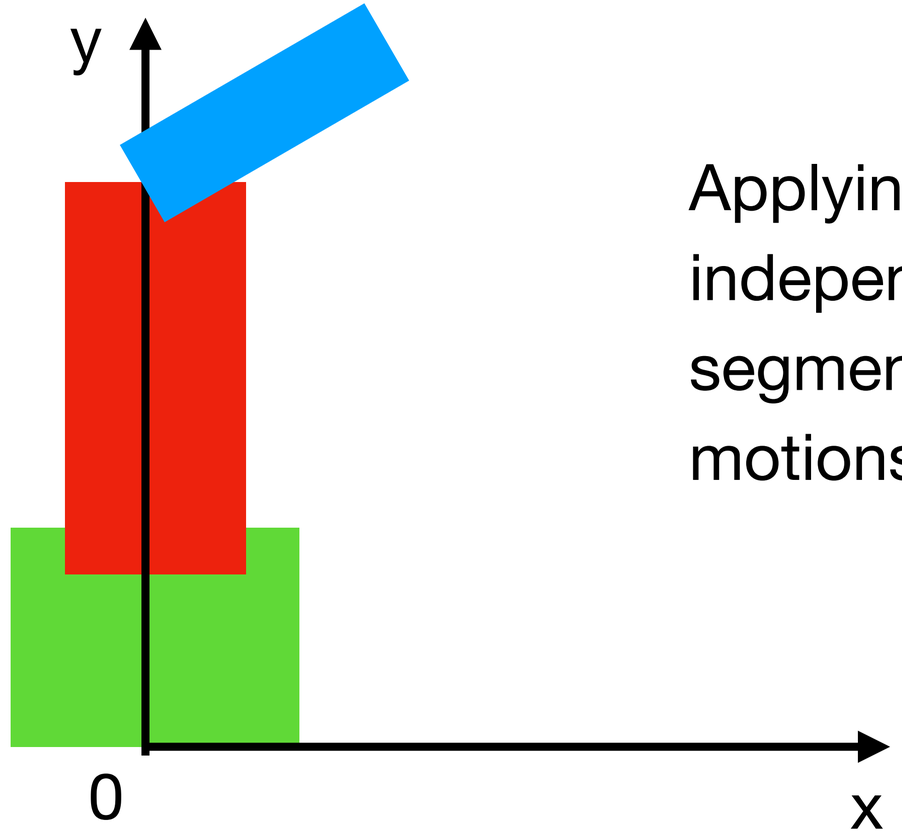
Independent Transformations

World coordinate frame

$T_2 \cdot R_2 \cdot S_2$

$T_1 \cdot S_1$

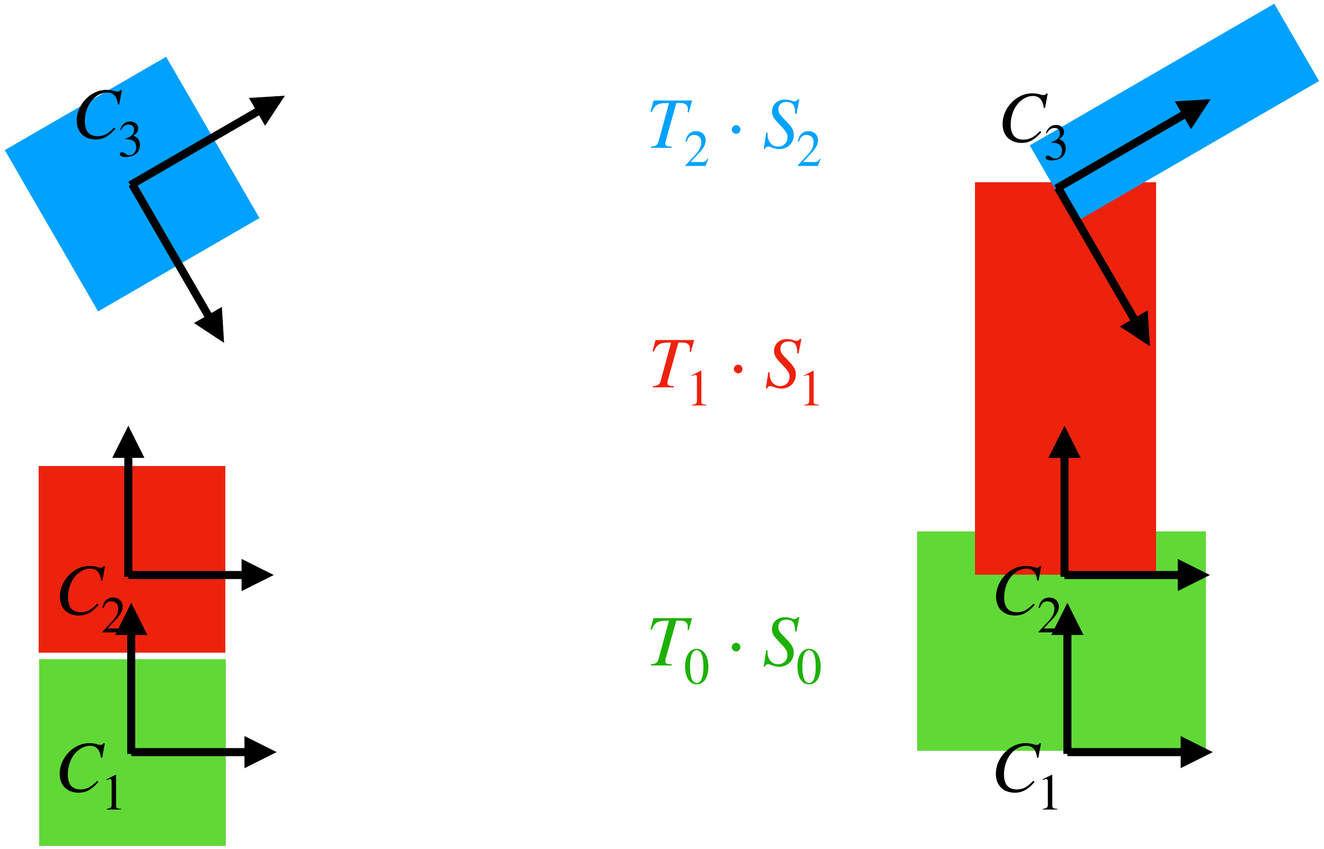
$T_0 \cdot S_0$



Applying transformations independently to each segment will not carry the motions along the chain

Hierarchical Transformations

Local coordinate frame



Hierarchical Transformations

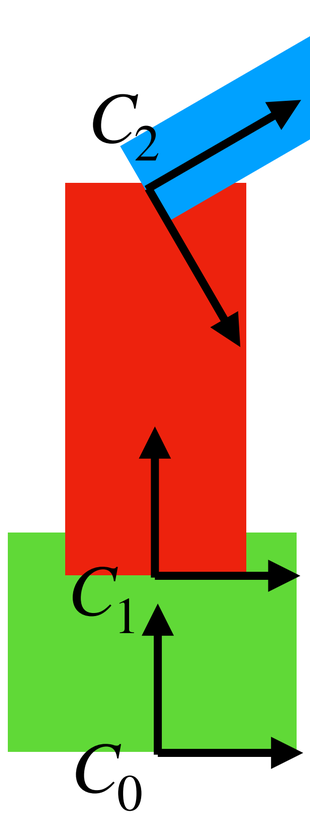
Segment transformations and coordinate transformations

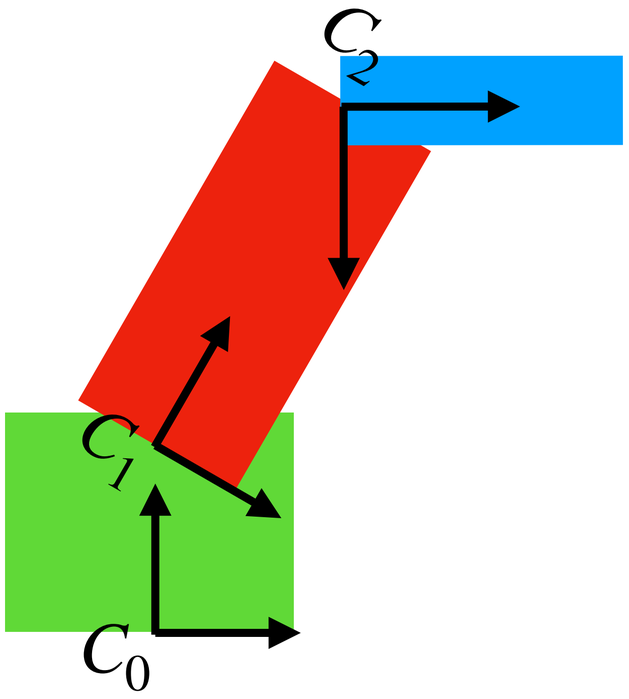
$M_{segment}$

$T_2 \cdot S_2$

$T_1 \cdot S_1$

$T_0 \cdot S_0$





$M_{coordinate}$

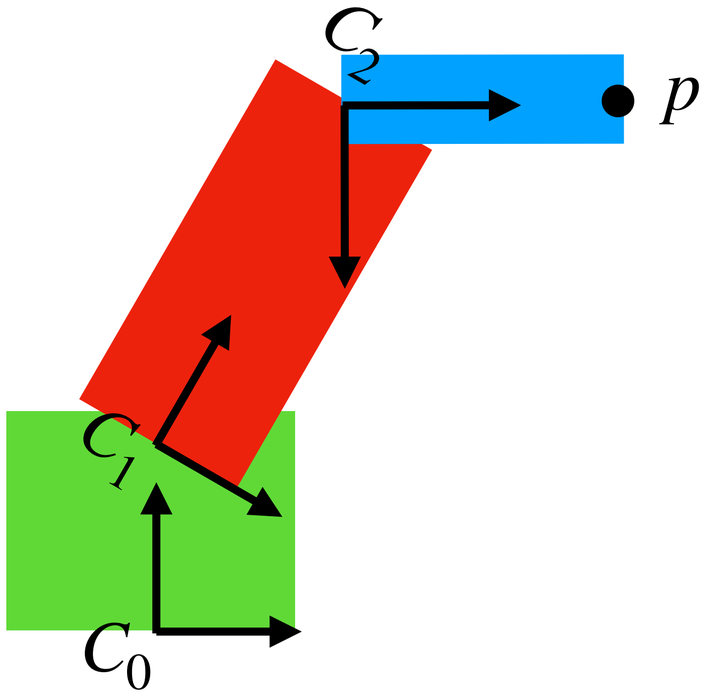
$T_{C_2} \cdot R_{C_2}$

$T_{C_1} \cdot R_{C_1}$

T_{C_0}

Positioned and oriented relative to its parent frame in a hierarchy

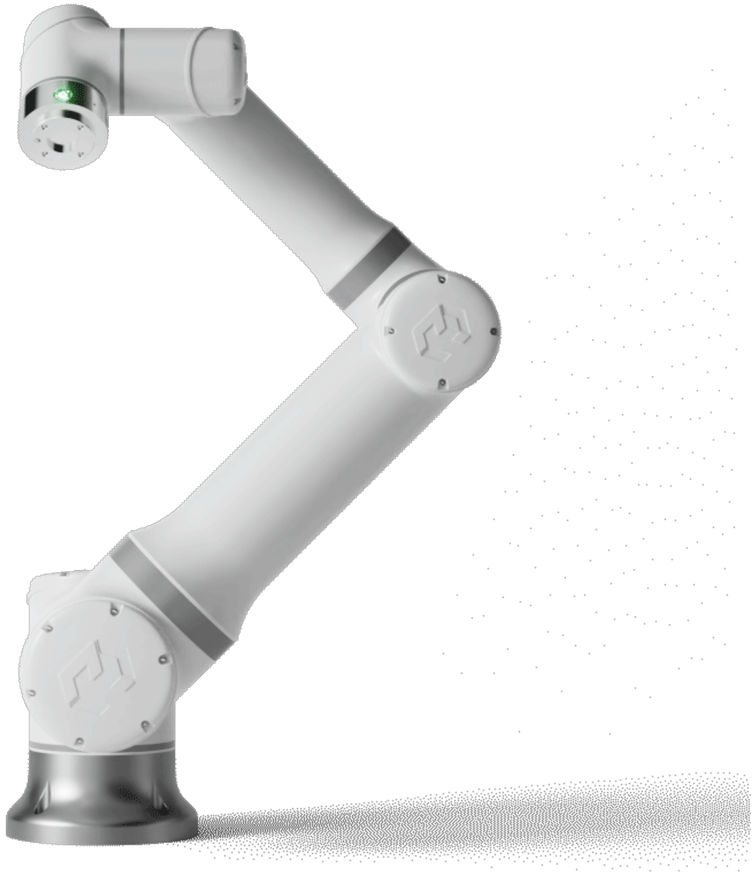
Forward Kinematics



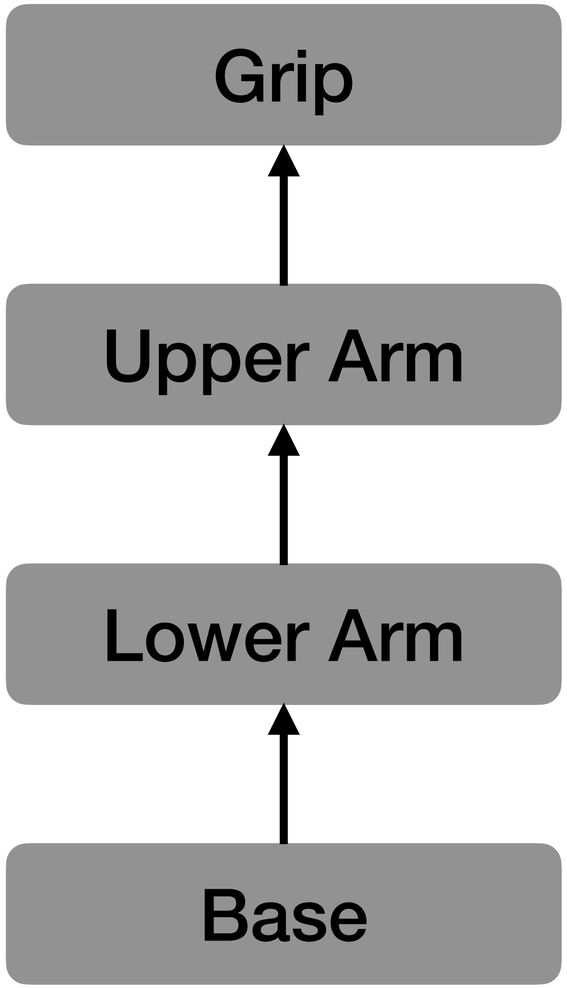
$$p_{world} = M_{w0} \cdot M_{01} \cdot M_{12} \cdot p$$

Hierarchical Modeling

Forward Kinematics



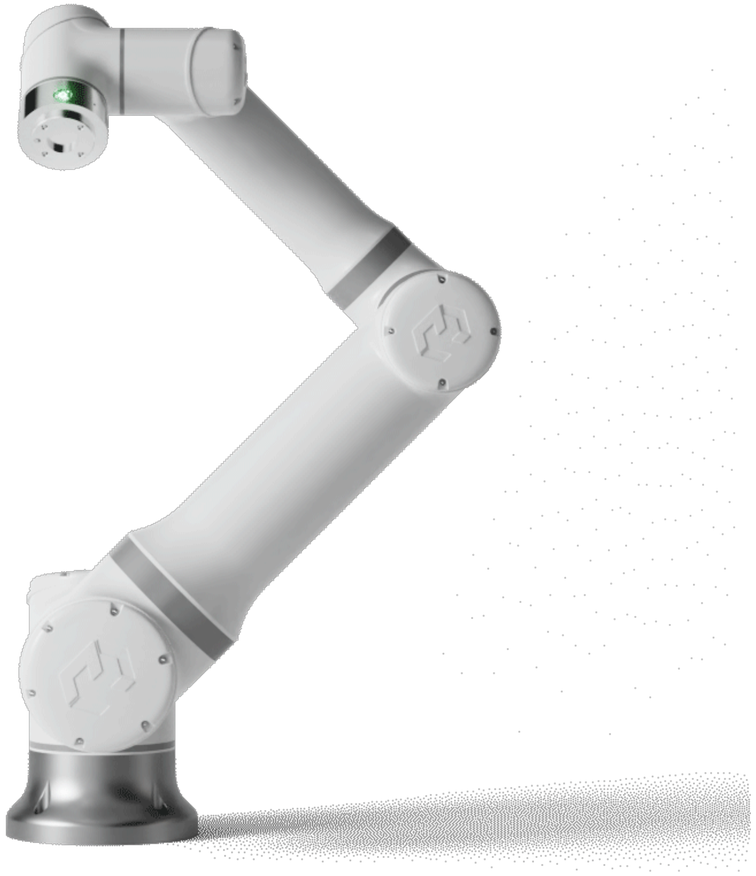
[source: Rozitek]



$$M_{world}(child) = M_{world}(parent) \cdot M_{local}(child)$$

Hierarchical Modeling

Forward Kinematics



[source: Rozitek]

$$M_{base} = T_{base} \cdot R_{base}$$

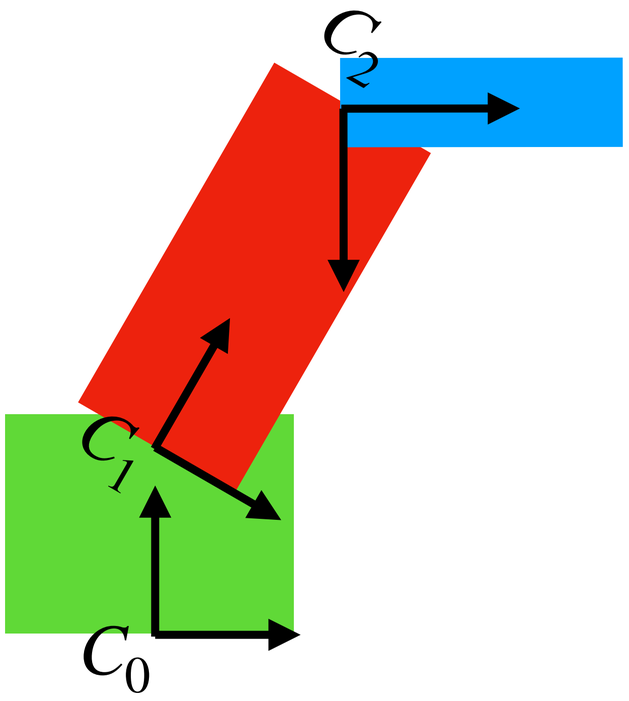
$$M_{lower} = M_{base} \cdot T_{lower} \cdot R_{lower}$$

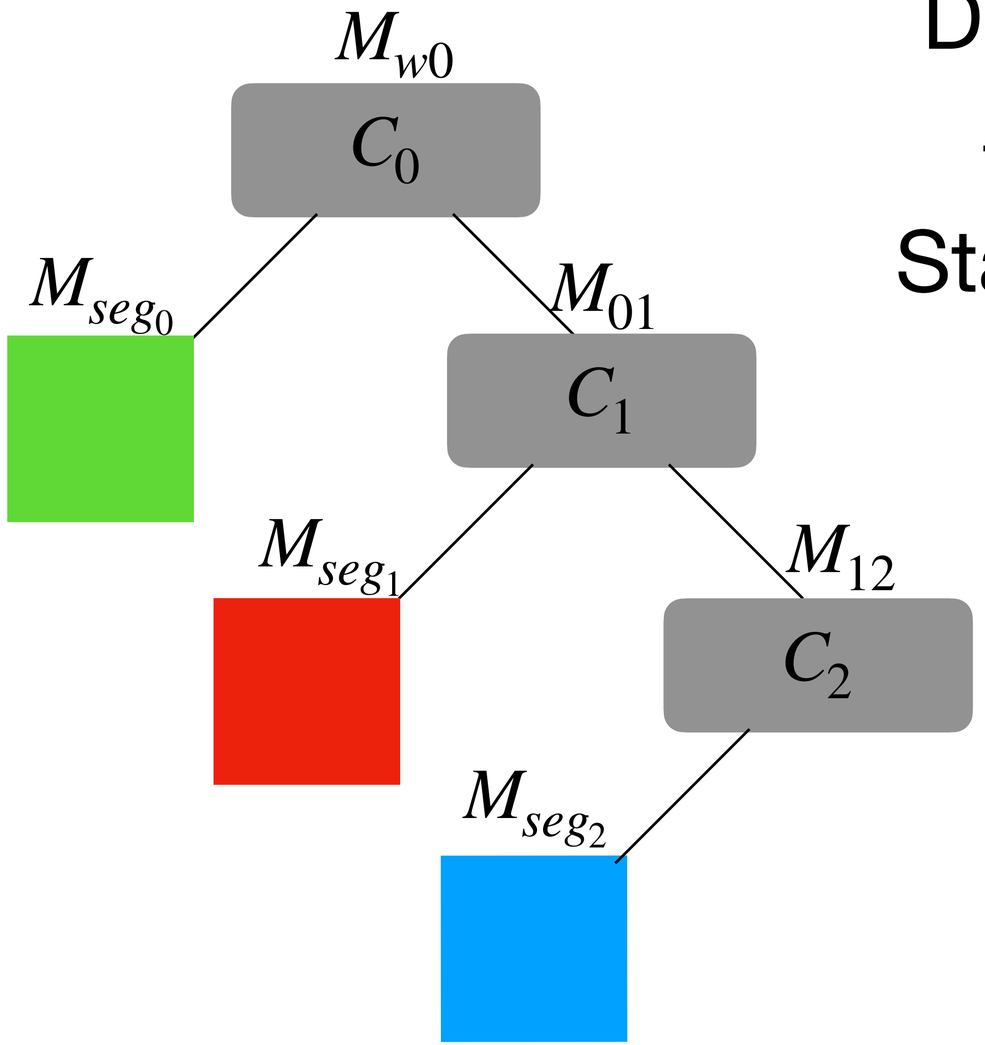
$$M_{upper} = M_{lower} \cdot T_{upper} \cdot R_{upper}$$

$$M_{grip} = M_{upper} \cdot T_{grip} \cdot R_{grip}$$

Data Structure

Tree and stack





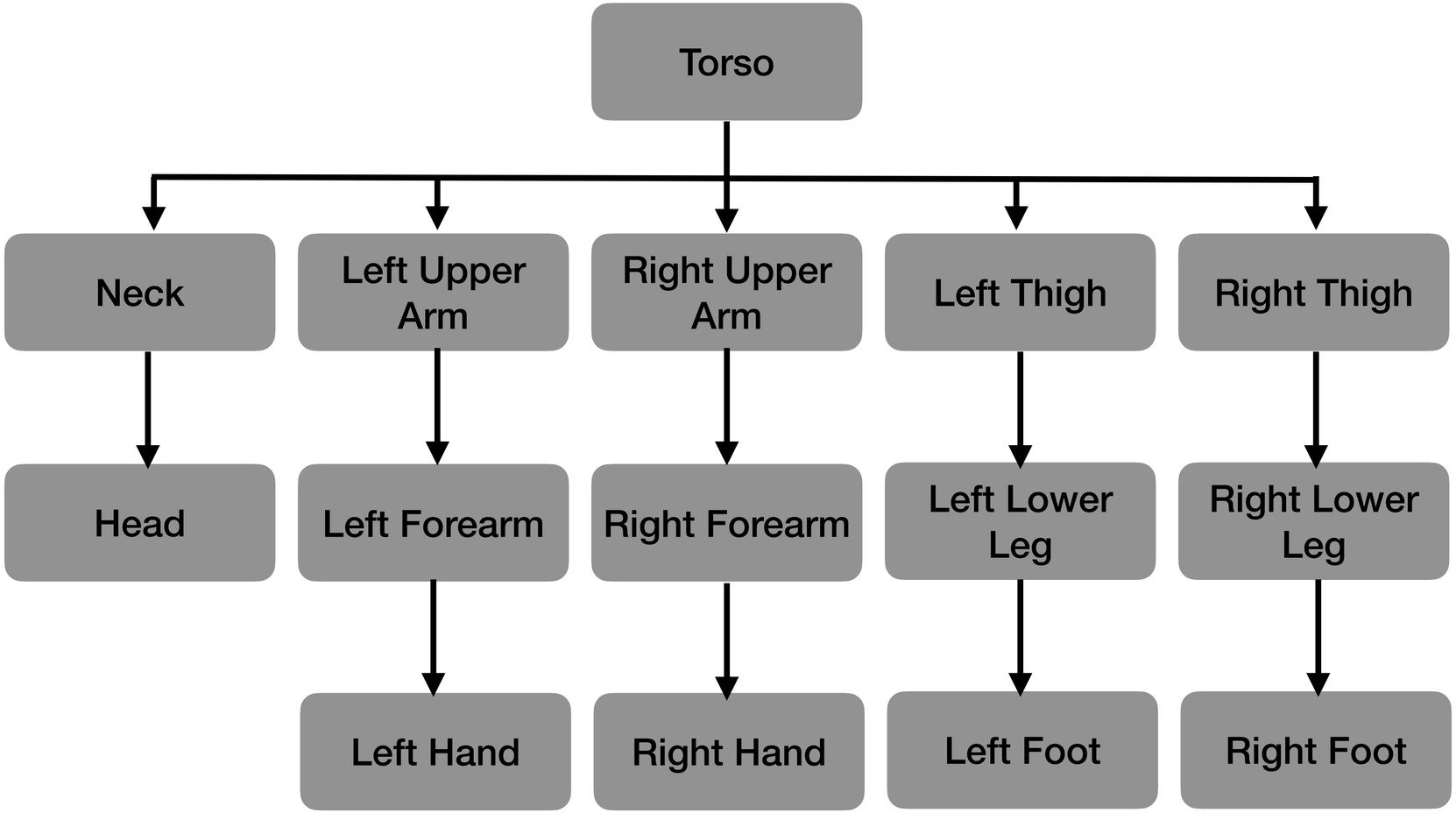
DFS

+

Stack

Hierarchical Modeling

Hierarchy of Humanoid robots



Hierarchical Modeling

Logic of the process

```
pushMatrix();        // world -> torso frame
        applyFrameTransform(Torso);
        drawTorso();

        pushMatrix();    // torso -> neck frame
                applyFrameTransform(Neck);
                drawNeck();

                pushMatrix();      // neck -> head frame
                        applyFrameTransform(Head);
                        drawHead();
                popMatrix();       restore neck frame
        popMatrix();    // restore torso frame

                        // same for limbs
                                ...
popMatrix();    //restore world frame
```